Apache Spark

Data Science · Machine Learning · AI Engineering

A Comprehensive Technical Reference

Covering Core Architecture, MLlib, Structured Streaming,
Deep Learning, Feature Stores, and LLM Pipelines

Spark 3.5+ | PySpark | Scala | AI Engineering

April 16, 2026

Apache Spark is an open-source unified analytics engine for large-scale data processing.
This document covers its full stack from core architecture to modern AI engineering.

Contents

1 Introduction to Apache Spark **2**
1.1 Core Design Principles . . . 2
1.2 Spark vs. Hadoop MapReduce . . . 2

2 Architecture **2**
2.1 Cluster Architecture . . . 2
2.1.1 Driver . . . 2
2.1.2 Cluster Managers . . . 3
2.1.3 Executors . . . 3
2.2 DAG Execution Model . . . 3
2.3 Memory Management . . . 3

3 Core APIs **4**
3.1 Resilient Distributed Datasets (RDDs) . . . 4
3.2 DataFrame API . . . 4
3.3 Spark SQL . . . 5
3.4 Catalyst Optimizer and Tungsten . . . 5

4 Data Engineering for Data Science **5**
4.1 Production ETL Pipelines . . . 5
4.2 Feature Engineering with Window Functions . . . 6
4.3 Lakehouse Formats . . . 7

5 MLlib — Machine Learning **7**
5.1 The Pipeline API . . . 7
5.2 Supervised Learning Algorithms . . . 8
5.3 Unsupervised Learning . . . 8
5.4 Collaborative Filtering (ALS) . . . 8

6 Structured Streaming **9**
6.1 Core Concepts . . . 9
6.2 Kafka Streaming with Real-Time Inference . . . 9

7 AI Engineering Integrations **10**
7.1 Distributed Deep Learning (Horovod) . . . 10
7.2 MLflow Experiment Tracking . . . 10
7.3 Feature Stores . . . 11
7.4 LLM Pipelines on Spark . . . 11
7.5 Pandas API on Spark . . . 12

8 Graph Analytics **12**
8.1 GraphFrames . . . 12

9 Storage and Data Sources **12**
9.1 File Format Comparison . . . 13
9.2 Connector Examples . . . 13

10 Deployment and Operations **13**
10.1 Submitting to Kubernetes . . . 13
10.2 Key Configuration Reference . . . 14
10.3 Performance Tuning Checklist . . . 14

11 Language Interfaces **14**

12 Quick Reference Summary **15**

1 Introduction to Apache Spark

Apache Spark is an open-source, distributed computing system for fast, large-scale data processing. Originally developed at UC Berkeley's AMPLab in 2009 and donated to the Apache Software Foundation in 2013, Spark has become the *de facto* standard for data engineering, data science, and machine learning at scale.

Why Spark?

Spark can be up to **100× faster** than Hadoop MapReduce for in-memory workloads and 10× faster for disk-based tasks. Its unified programming model lets teams write ETL pipelines, run SQL queries, train ML models, and process real-time streams—all in a single framework and a single cluster.

1.1 Core Design Principles

- **Speed** — In-memory computation, DAG execution engine, Tungsten code generation
- **Ease of use** — High-level APIs in Python, Scala, Java, R, and SQL
- **Generality** — Batch, streaming, ML, and graph in one engine
- **Fault tolerance** — RDD lineage-based recovery and checkpointing
- **Scalability** — Runs on a laptop or a 10,000-node cloud cluster

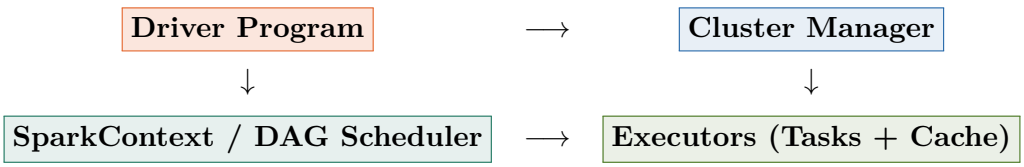
1.2 Spark vs. Hadoop MapReduce

Aspect	Apache Spark	Hadoop MapReduce
Processing	In-memory DAG	Disk-based Map + Reduce
Speed	Up to 100× faster	Baseline
APIs	Python, Scala, SQL, R	Java only (natively)
Streaming	Structured Streaming built-in	Requires external tools
ML support	MLlib built-in	Mahout (deprecated)
Ease of use	High	Low

2 Architecture

2.1 Cluster Architecture

A Spark application runs as independent processes coordinated by a **SparkContext** in the driver program.



2.1.1 Driver

The driver process hosts the main function and:

- Creates and owns the `SparkSession` / `SparkContext`
- Builds the logical plan and converts it to a DAG

- Schedules tasks on executors via the DAG Scheduler
- Collects results and returns them to the user

2.1.2 Cluster Managers

Manager	Description
Standalone	Built-in simple cluster manager; easy to set up for dedicated Spark clusters
YARN	Hadoop resource manager; widely used in on-premises Hadoop ecosystems
Kubernetes	Container-native; preferred for cloud-native and multi-tenant deployments
Mesos	Fine-grained resource sharing; now less common
Local	Single-machine mode; used for development, testing, and notebooks

2.1.3 Executors

Executors are JVM processes on worker nodes. Each executor:

- Runs tasks assigned by the driver in parallel
- Caches data in memory or on disk (configurable per RDD/DataFrame)
- Has a configurable number of CPU cores and heap memory

2.2 DAG Execution Model

Spark transforms user operations into a **Directed Acyclic Graph** (DAG):

1. **Transformations** are lazy—they build the plan without executing: `map`, `filter`, `join`, `groupBy`
2. **Actions** trigger execution of the full DAG: `collect`, `count`, `save`, `show`
3. The DAG Scheduler splits the graph into **stages** at shuffle boundaries
4. The Task Scheduler assigns one **task** per partition to available executors

Lazy evaluation allows Spark to optimize the entire query plan before running it, enabling Catalyst to push down filters, reorder joins, and generate efficient bytecode via the Tungsten engine.

2.3 Memory Management

Spark uses a unified memory model with four regions:

- **Execution memory** — Shuffles, joins, sorts, aggregations
- **Storage memory** — Caching DataFrames and RDDs (`df.cache()`)
- **User memory** — User data structures and UDFs
- **Reserved memory** — System overhead (300 MB, fixed)

Key config: `spark.memory.fraction` (default 0.6) controls the execution + storage pool as a fraction of JVM heap.

3 Core APIs

3.1 Resilient Distributed Datasets (RDDs)

RDDs are Spark's original low-level abstraction: immutable, partitioned collections of records distributed across the cluster.

Listing 1: Basic RDD operations

```python
from pyspark import SparkContext

sc = SparkContext("local[*]", "RDD Demo")
rdd = sc.parallelize(range(1, 11), numSlices=4)

squared = rdd.map(lambda x: x ** 2)        # transformation (lazy)
evens   = rdd.filter(lambda x: x % 2 == 0) # transformation (lazy)

# Word count with key-value RDD
words = sc.textFile("hdfs://data/corpus.txt")
counts = (words
    .flatMap(lambda line: line.split())
    .map(lambda word: (word, 1))
    .reduceByKey(lambda a, b: a + b)
    .sortBy(lambda kv: kv[1], ascending=False)
)
results = counts.take(20)  # action: triggers execution
```

3.2 DataFrame API

DataFrames are distributed collections of data organized into named columns. They sit on top of RDDs and benefit from Catalyst optimization and Tungsten execution.

Listing 2: DataFrame operations in PySpark

```python
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.types import (
    StructType, StructField, StringType, IntegerType, DoubleType
)

spark = (SparkSession.builder
    .appName("DataFrame Demo")
    .config("spark.sql.adaptive.enabled", "true")
    .getOrCreate())

schema = StructType([
    StructField("user_id",  StringType(),  True),
    StructField("age",      IntegerType(), True),
    StructField("amount",   DoubleType(),  True),
    StructField("category", StringType(),  True),
])

df = spark.read.schema(schema).parquet("s3://bucket/events/")

result = (df
    .filter(F.col("age") >= 18)
    .withColumn("amount_log", F.log1p("amount"))
    .withColumn("age_group",
        F.when(F.col("age") < 30, "young")
         .when(F.col("age") < 50, "mid")
         .otherwise("senior"))
    .groupBy("category", "age_group")
    .agg(
```

```
        F.count("*").alias("n"),
        F.mean("amount").alias("avg_amount"),
        F.stddev("amount").alias("std_amount"),
        F.percentile_approx("amount", 0.5).alias("median"),
    )
    .orderBy(F.desc("avg_amount"))
)

result.write.mode("overwrite").parquet("s3://bucket/output/")
```

3.3 Spark SQL

Listing 3: SQL queries over DataFrames

```
df.createOrReplaceTempView("transactions")

result = spark.sql("""
    SELECT
        category,
        age_group,
        COUNT(*)                          AS n,
        ROUND(AVG(amount), 2)             AS avg_amount,
        PERCENTILE_APPROX(amount, 0.5)    AS median_amount
    FROM (
        SELECT *,
            CASE WHEN age < 30 THEN 'young'
                 WHEN age < 50 THEN 'mid'
                 ELSE 'senior' END AS age_group
        FROM transactions
        WHERE age >= 18
    ) t
    GROUP BY category, age_group
    ORDER BY avg_amount DESC
""")
```

3.4 Catalyst Optimizer and Tungsten

- **Analysis** — Resolves column names, functions, and types
- **Logical optimization** — Predicate pushdown, constant folding, projection pruning
- **Physical planning** — Chooses join strategies: broadcast, sort-merge, shuffle-hash
- **Code generation** — Tungsten generates optimized JVM bytecode at runtime

4 Data Engineering for Data Science

4.1 Production ETL Pipelines

Listing 4: ETL pipeline with Delta Lake upsert

```
from delta import DeltaTable

def run_etl(spark, input_path, output_path):
    # EXTRACT
    raw = (spark.read
        .option("header", True)
        .option("inferSchema", False)
        .csv(input_path))

    # TRANSFORM
```

```python
    cleaned = (raw
        .dropDuplicates(["user_id", "event_date"])
        .filter(F.col("event_date").isNotNull())
        .withColumn("event_ts",
            F.to_timestamp("event_date", "yyyy-MM-dd"))
        .withColumn("amount", F.col("amount").cast(DoubleType()))
        .fillna({"category": "unknown", "country": "XX"}))

    enriched = (cleaned
        .withColumn("dow",        F.dayofweek("event_ts"))
        .withColumn("hour",       F.hour("event_ts"))
        .withColumn("is_weekend", F.col("dow").isin([1,7]).cast("int")))

    # LOAD (Delta upsert / merge)
    if DeltaTable.isDeltaTable(spark, output_path):
        delta = DeltaTable.forPath(spark, output_path)
        (delta.alias("tgt")
              .merge(enriched.alias("src"),
                     "tgt.user_id = src.user_id "
                     "AND tgt.event_ts = src.event_ts")
              .whenMatchedUpdateAll()
              .whenNotMatchedInsertAll()
              .execute())
    else:
        (enriched.write
            .format("delta")
            .mode("overwrite")
            .partitionBy("event_date")
            .save(output_path))
```

4.2 Feature Engineering with Window Functions

Listing 5: Rolling aggregations and lag features

```python
from pyspark.sql import Window

user_window = (Window.partitionBy("user_id")
                     .orderBy("event_ts"))
rolling_7d  = (Window.partitionBy("user_id")
                     .orderBy(F.unix_timestamp("event_ts"))
                     .rangeBetween(-7*86400, 0))
rolling_30d = (Window.partitionBy("user_id")
                     .orderBy(F.unix_timestamp("event_ts"))
                     .rangeBetween(-30*86400, 0))

features = (df
    .withColumn("prev_amount",  F.lag("amount", 1).over(user_window))
    .withColumn("amount_delta",
        F.col("amount") - F.col("prev_amount"))
    .withColumn("sum_7d",       F.sum("amount").over(rolling_7d))
    .withColumn("count_7d",     F.count("*").over(rolling_7d))
    .withColumn("avg_30d",      F.avg("amount").over(rolling_30d))
    .withColumn("stddev_30d",   F.stddev("amount").over(rolling_30d))
    .withColumn("purchase_rank",F.rank().over(user_window))
    .withColumn("pct_rank",     F.percent_rank().over(user_window)))
```

4.3 Lakehouse Formats

Format	Key Features	Best For
Delta Lake	ACID, time travel, Z-order, schema enforcement	Databricks, unified batch and stream
Apache Iceberg	Hidden partitioning, table evolution, multi-engine	Multi-engine environments (Flink, Trino, Spark)
Apache Hudi	Upserts, record-level indexing, incremental queries	Near-real-time CDC pipelines

5 MLlib — Machine Learning

What is MLlib?

MLlib is Spark's distributed machine learning library. It provides scalable implementations of common algorithms, a rich set of feature transformers, and a Pipeline API inspired by scikit-learn. All training runs natively on the cluster — no data needs to leave Spark.

5.1 The Pipeline API

Listing 6: End-to-end ML pipeline with cross-validation

```python
from pyspark.ml import Pipeline
from pyspark.ml.feature import (
    StringIndexer, OneHotEncoder,
    VectorAssembler, StandardScaler, Imputer
)
from pyspark.ml.classification import GBTClassifier
from pyspark.ml.evaluation import BinaryClassificationEvaluator
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder

imputer     = Imputer(inputCols=["age", "amount"],
                      outputCols=["age_imp", "amount_imp"])
cat_indexer = StringIndexer(inputCol="category",
                            outputCol="cat_idx",
                            handleInvalid="keep")
encoder     = OneHotEncoder(inputCols=["cat_idx"],
                           outputCols=["cat_ohe"])
assembler   = VectorAssembler(
    inputCols=["age_imp", "amount_imp",
               "cat_ohe", "sum_7d", "avg_30d", "is_weekend"],
    outputCol="features_raw")
scaler      = StandardScaler(inputCol="features_raw",
                             outputCol="features",
                             withStd=True, withMean=True)
gbt         = GBTClassifier(featuresCol="features",
                            labelCol="label",
                            maxIter=100, maxDepth=6,
                            stepSize=0.05, subsamplingRate=0.8,
                            seed=42)

pipeline = Pipeline(stages=[
    imputer, cat_indexer, encoder, assembler, scaler, gbt])

param_grid = (ParamGridBuilder()
    .addGrid(gbt.maxDepth, [4, 6, 8])
    .addGrid(gbt.maxIter,  [50, 100])
    .addGrid(gbt.stepSize, [0.05, 0.1])
    .build())
```

```python

cv = CrossValidator(
    estimator=pipeline,
    estimatorParamMaps=param_grid,
    evaluator=BinaryClassificationEvaluator(
        metricName="areaUnderROC"),
    numFolds=5, seed=42, parallelism=4)

train_df, test_df = df.randomSplit([0.8, 0.2], seed=42)
cv_model = cv.fit(train_df)

predictions = cv_model.transform(test_df)
auc = BinaryClassificationEvaluator().evaluate(predictions)
print(f"Test AUC-ROC: {auc:.4f}")
```

5.2 Supervised Learning Algorithms

Algorithm	Task	Key Hyperparameters
Linear Regression	Regression	regParam, elasticNetParam
Logistic Regression	Classification	regParam, maxIter, threshold
Decision Tree	Both	maxDepth, maxBins
Random Forest	Both	numTrees, maxDepth, featureSubsetStrategy
Gradient Boosted Tree	Both	maxIter, stepSize, subsamplingRate
Linear SVM	Classification	maxIter, regParam
Naive Bayes	Classification	smoothing, modelType
Factorization Machines	Both	numFactors, stepSize

5.3 Unsupervised Learning

Listing 7: K-Means with silhouette-based model selection

```python
from pyspark.ml.clustering import KMeans
from pyspark.ml.evaluation import ClusteringEvaluator

evaluator = ClusteringEvaluator(metricName="silhouette")
best_k, best_score = 2, -1.0

for k in range(2, 16):
    km = KMeans(featuresCol="features", k=k, seed=42,
                maxIter=100)
    model = km.fit(features_df)
    preds = model.transform(features_df)
    score = evaluator.evaluate(preds)
    wssse = model.summary.trainingCost
    print(f"k={k:2d}  silhouette={score:.4f}  wssse={wssse:,.0f}")
    if score > best_score:
        best_k, best_score = k, score
```

5.4 Collaborative Filtering (ALS)

Listing 8: ALS recommendation system

```python
from pyspark.ml.recommendation import ALS

als = ALS(
    rank=100, maxIter=15, regParam=0.01,
    userCol="user_id", itemCol="item_id", ratingCol="rating",
```

```
    coldStartStrategy="drop", implicitPrefs=False,
    nonnegative=True, seed=42)

model = als.fit(train_df)
user_recs = model.recommendForAllUsers(10)
item_recs = model.recommendForAllItems(10)
```

6 Structured Streaming

Structured Streaming models a live data stream as a continuously growing unbounded table, using the same DataFrame API as batch processing.

6.1 Core Concepts

- **Micro-batch mode** — Default; each trigger interval is processed as a mini-batch
- **Continuous mode** — Low-latency (∼1 ms end-to-end); limited operations
- **Watermarking** — Handles late data with a configurable tolerance threshold
- **Output modes** — *Append*, *Complete*, *Update*
- **Checkpointing** — Fault-tolerant state recovery via write-ahead logs

6.2 Kafka Streaming with Real-Time Inference

Listing 9: Kafka stream with windowed aggregation and ML scoring

```
from pyspark.sql.types import StructType, StringType, DoubleType

schema = (StructType()
    .add("user_id", StringType())
    .add("amount",  DoubleType())
    .add("ts",      StringType()))

kafka_df = (spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "broker:9092")
    .option("subscribe", "transactions")
    .option("maxOffsetsPerTrigger", 100_000)
    .load())

parsed = (kafka_df
    .select(F.from_json(
        F.col("value").cast("string"), schema).alias("d"))
    .select("d.*")
    .withColumn("ts", F.to_timestamp("ts")))

windowed = (parsed
    .withWatermark("ts", "10 minutes")
    .groupBy(F.window("ts", "5 minutes", "1 minute"), "user_id")
    .agg(F.sum("amount").alias("total"),
         F.count("*").alias("tx_count"),
         F.max("amount").alias("max_amount")))

from pyspark.ml import PipelineModel
model = PipelineModel.load("s3://models/fraud_detector_v3/")
scored = model.transform(windowed)

query = (scored.writeStream
    .format("delta")
    .option("checkpointLocation", "s3://checkpoints/fraud/")
```

```
    .outputMode("append")
    .trigger(processingTime="30 seconds")
    .start("s3://output/fraud_scores/"))

query.awaitTermination()
```

7 AI Engineering Integrations

Spark in the Modern AI Stack

Spark serves as the **data backbone** of AI engineering: petabyte-scale feature computation, distributed deep learning, batch LLM inference, and real-time scoring pipelines all rely on Spark for scale that single-node tools cannot match.

7.1 Distributed Deep Learning (Horovod)

Listing 10: Distributed PyTorch training with Horovod on Spark

```python
import horovod.spark.torch as hvd
from horovod.spark.common.backend import SparkBackend
from horovod.spark.common.store import HDFSStore
import torch.nn as nn, torch

class FraudNet(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.BatchNorm1d(256), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(256, 128), nn.ReLU(),
            nn.Linear(128, 1),  nn.Sigmoid())
    def forward(self, x):
        return self.net(x)

backend = SparkBackend(num_proc=8)  # 8 GPU workers
store   = HDFSStore("/models/checkpoints")

estimator = hvd.TorchEstimator(
    backend=backend, store=store,
    model=FraudNet(input_dim=64),
    optimizer=lambda m: torch.optim.Adam(m.parameters(), lr=1e-3),
    loss=nn.BCELoss, epochs=50, batch_size=1024,
    feature_cols=feature_cols, label_cols=["label"])

model = estimator.fit(train_df)
```

7.2 MLflow Experiment Tracking

Listing 11: MLflow integration: tracking, registry, and promotion

```python
import mlflow, mlflow.spark
from mlflow.tracking import MlflowClient

mlflow.set_tracking_uri("https://mlflow.company.com")
mlflow.set_experiment("fraud-detection-v2")

with mlflow.start_run(run_name="GBT_baseline") as run:
    mlflow.log_params({
        "max_depth": 6, "max_iter": 100, "step_size": 0.05,
        "train_rows": train_df.count(),
```

```
        "n_features": len(feature_cols),
    })

    model = pipeline.fit(train_df)
    preds = model.transform(test_df)
    auc   = BinaryClassificationEvaluator(
        metricName="areaUnderROC").evaluate(preds)
    mlflow.log_metrics({"auc_roc": auc})

    from mlflow.models import infer_signature
    sig = infer_signature(train_df.drop("label"),
                          preds.select("prediction"))
    mlflow.spark.log_model(model, "model", signature=sig)

    client = MlflowClient()
    mv = client.create_model_version(
        name="fraud-detector",
        source=f"runs:/{run.info.run_id}/model",
        run_id=run.info.run_id)
    client.transition_model_version_stage(
        name="fraud-detector",
        version=mv.version,
        stage="Production")
```

7.3 Feature Stores

Feature stores solve the training/serving skew problem by providing a single source of truth for features used in both offline training and online real-time scoring.

Feature Store	Company / Origin	Key Capability
Feast	Open-source	Lightweight, supports Spark batch materialization
Tecton	Tecton.ai	Enterprise; real-time + batch; Spark SQL feature views
Databricks	Databricks	Unity Catalog integrated; online + offline tables
Vertex FS	Google Cloud	GCP native; BigQuery + Bigtable backends

7.4 LLM Pipelines on Spark

Listing 12: Batch embedding generation for RAG at scale

```
from pyspark.sql.types import ArrayType, FloatType
import openai, os

@F.udf(returnType=ArrayType(FloatType()))
def embed_text(text):
    if not text:
        return None
    client = openai.OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
    resp = client.embeddings.create(
        model="text-embedding-3-small",
        input=text[:8192])
    return resp.data[0].embedding

embedded = (documents_df
    .repartition(200)               # control API parallelism
```

```
    .withColumn("embedding", embed_text(F.col("text")))
    .filter(F.col("embedding").isNotNull()))

embedded.write.format("delta").save("s3://vector-store/embeddings/")
```

7.5 Pandas API on Spark

Listing 13: pyspark.pandas — familiar syntax at distributed scale

```
import pyspark.pandas as ps

psdf = ps.read_parquet("s3://data/large_dataset/")
psdf["amount_sqrt"] = psdf["amount"].apply(
    lambda x: x**0.5 if x > 0 else 0)

summary = psdf.groupby("category")["amount"].agg(
    ["mean", "std", "count"])

# Apply per-group normalization
def normalize(group):
    mu = group["amount"].mean()
    sd = group["amount"].std() + 1e-9
    group["amount_z"] = (group["amount"] - mu) / sd
    return group

result   = psdf.groupby("category").apply(normalize)
spark_df = result.to_spark()   # back to Spark DataFrame
```

8 Graph Analytics

8.1 GraphFrames

Listing 14: Social network analysis with GraphFrames

```
from graphframes import GraphFrame

vertices = spark.createDataFrame([
    ("u1","Alice"), ("u2","Bob"), ("u3","Carol")
], ["id","name"])

edges = spark.createDataFrame([
    ("u1","u2","follows"), ("u2","u3","follows"),
    ("u3","u1","follows"),
], ["src","dst","relationship"])

g = GraphFrame(vertices, edges)

pr = g.pageRank(resetProbability=0.15, maxIter=10)
pr.vertices.orderBy("pagerank", ascending=False).show()

cc  = g.connectedComponents()
tc  = g.triangleCount()
sp  = g.shortestPaths(landmarks=["u1","u3"])

# Structural motif finding
triangles = g.find("(a)-[e1]->(b); (b)-[e2]->(c); (c)-[e3]->(a)")
```

9 Storage and Data Sources

9.1 File Format Comparison

Format	Encoding	Splittable	Best Use Case
Parquet	Columnar	Yes	Analytics, ML features, archival
ORC	Columnar	Yes	Hive-heavy ecosystems
Delta	Columnar	Yes	ACID lakehouse tables with DML
Avro	Row	Yes	Schema evolution, Kafka serialization
JSON	Row	No	APIs, semi-structured exploratory data
CSV	Row	Yes	Human-readable, simple ingestion

9.2 Connector Examples

Listing 15: Reading from multiple source systems

```python
# Amazon S3
df_s3 = spark.read.parquet("s3a://bucket/events/")

# PostgreSQL over JDBC with parallel reads
df_pg = (spark.read
    .format("jdbc")
    .option("url", "jdbc:postgresql://host:5432/db")
    .option("dbtable", "public.events")
    .option("numPartitions", 32)
    .option("partitionColumn", "id")
    .option("lowerBound", 1)
    .option("upperBound", 10_000_000)
    .load())

# Delta Lake
df_delta = spark.read.format("delta").load("s3://lakehouse/events/")

# Kafka batch read
df_kafka = (spark.read
    .format("kafka")
    .option("kafka.bootstrap.servers", "broker:9092")
    .option("subscribe", "events")
    .option("startingOffsets", "earliest")
    .load())
```

10 Deployment and Operations

10.1 Submitting to Kubernetes

Listing 16: spark-submit to a Kubernetes cluster

```bash
spark-submit \
  --master k8s://https://k8s-api:6443 \
  --deploy-mode cluster \
  --name fraud-training \
  --conf spark.executor.instances=20 \
  --conf spark.executor.memory=16g \
  --conf spark.executor.cores=4 \
  --conf spark.driver.memory=4g \
  --conf spark.kubernetes.container.image=registry/spark:3.5.0 \
  --conf spark.kubernetes.namespace=data-science \
  --conf spark.sql.adaptive.enabled=true \
  --conf spark.sql.adaptive.skewJoin.enabled=true \
  --conf spark.eventLog.enabled=true \
  --conf spark.eventLog.dir=s3a://logs/spark-events/ \
  s3a://jobs/fraud_training_v2.py
```

10.2 Key Configuration Reference

Configuration Key	Default	Purpose
`spark.sql.adaptive.enabled`	true	Adaptive Query Execution (AQE)
`spark.sql.adaptive.skewJoin.enabled`	true	Automatically fix data skew in joins
`spark.memory.fraction`	0.6	Fraction of JVM heap for Spark pools
`spark.sql.shuffle.partitions`	200	Partitions after a shuffle operation
`spark.default.parallelism`	cores	Default RDD parallelism
`spark.serializer`	Java	Kryo for 10x serialization speedup
`spark.sql.files.maxPartitionBytes`	128 MB	Maximum partition size when reading files
`spark.dynamicAllocation.enabled`	false	Auto-scale executor count

10.3 Performance Tuning Checklist

1. Enable Adaptive Query Execution (`spark.sql.adaptive.enabled=true`)
2. Use Parquet or Delta instead of CSV/JSON
3. Cache DataFrames used multiple times (`df.cache()` or `df.persist()`)
4. Avoid `collect()` on large DataFrames; use `take(n)` or write to storage
5. Tune `spark.sql.shuffle.partitions` — rule of thumb: 2–3× total cores
6. Broadcast small lookup tables (`F.broadcast(small_df)`)
7. Partition data by common filter and join keys
8. Inspect physical plans with `df.explain(mode="formatted")`
9. Switch to Kryo serializer for RDD-heavy code
10. Monitor the Spark UI for data skew, spill to disk, and GC pressure

11 Language Interfaces

Language	Strengths	Primary Audience
PySpark	Richest DS ecosystem; NumPy, pandas, sklearn interop	Data scientists, ML engineers
Scala	Native JVM; highest performance; type safety	Data and platform engineers
Java	Enterprise integration; verbose but stable	Backend and Java-native teams
R (SparkR)	GLMs, tidyverse-style; statistical workflows	Statisticians, quant analysts
SQL	Ad-hoc analysis, BI tools, fast aggregations	Analysts, everyone

12 Quick Reference Summary

Domain	Spark Component	Key APIs and Classes
Batch processing	DataFrame API	spark.read, DataFrame, functions
SQL queries	Spark SQL	spark.sql(), createOrReplaceTempView
Streaming	Struct. Streaming	readStream, writeStream, trigger
ML training	MLlib Pipeline	Pipeline, CrossValidator, ParamGridBuilder
Deep learning	Horovod / TF / PyTorch	hvd.TorchEstimator, KerasEstimator
Experiment tracking	MLflow	mlflow.start_run, mlflow.spark.log_model
Feature store	Feast / Tecton / FS	get_historical_features
Graph analytics	GraphFrames	GraphFrame, pageRank, find
Lakehouse	Delta Lake	DeltaTable, merge, time travel
Pandas compat	pandas API on Spark	pyspark.pandas

Recommended Learning Path:

1. Master the DataFrame API and Spark SQL
2. Understand lazy evaluation, the DAG, and shuffle mechanics
3. Build ETL pipelines with Delta Lake
4. Learn the MLlib Pipeline API and cross-validation patterns
5. Explore Structured Streaming for real-time workloads
6. Integrate MLflow for experiment tracking and model registry
7. Add feature stores and Horovod for full AI engineering workflows

Apache Spark is a trademark of the Apache Software Foundation.